Algorithmic Optimization for H.264 Deblocking Filter on Portable Devices

J. Ren[1] and N. Kehtarnavaz[1] ,Senior IEEE Member

[1]Department of Electrical Engineering, University of Texas at Dallas

Abstract — *Deblocking filters are used to reduce or eliminate blocking artifacts that appear when H.264/AVC is utilized for low bit rate video coding applications such as video conferencing, Considering that the H.264/AVC deblocking filter constitutes about one-third of the decoding time, in this paper, we present two optimization steps to reduce the computational complexity of this filter. These steps involve using the mode information of decoded macroblocks and the similarity among adjacent blocks. Experimental results show that these optimization steps significantly reduce the computational complexity with tolerable loss in video quality.*

Index Terms — Optimization of H.264/AVC deblocking filter, mode selection in video deblocking, block similarity in video deblocking.

I. INTRODUCTION

Due to the code efficiency superiority of the H.264/AVC video coding standard over the previous video coding standards, its use has been growing in consumer electronic products for applications such as video telephony, video conferencing and video surveillance. The code efficiency superiority is gained through added features and additional functionalities including variable block size mode selection, quarter pixel motion compensation and deblocking. However, such new added functionalities in H.264/AVC have greatly increased its computational complexity, which has affected its deployment on portable devices such as cell phones and PDAs. Often portable devices are used for decoding video streaming and playing back video clips. Thus, the optimization of the decoding algorithm constitutes a key step towards the successful deployment of H.264/AVC on portable devices.

Figure 1 shows a decoding diagram of H.264/AVC. An exact computational time or complexity for each sub-block shown in this figure is not possible because of the existence of different video content, video resolution and bit rate. However, one can obtain the time distribution averaged over many test sequences. In [1], such times are reported as follows: loop filtering (33% time), interpolation (25%), followed by bit-stream parsing and entropy decoding (13%) and reconstruction (13%). As can be seen among the sub-blocks, loop filtering has the largest computational complexity.

For low bit rate video decoding applications, blocking artifacts appear as H.264/AVC utilizes block transform video coding similar to the previous standards. In order to reduce or eliminate the blocking artifacts, several different deblocking algorithms [2, 3, 4, 5] have been proposed in the literature. Generally speaking, these algorithms can be classified into three categories: projection onto convex set approach [2], weighted sum of pixel across block boundary approach [3] and adaptive deblocking approach [4, 5]. In [6], we have provided the comparison of these algorithms in terms of computational complexity and thus their power consumption.

In [3], a deblocking filter is proposed using a weighted sum of quartets which are symmetrically aligned with image boundary. Different strategies are applied in the monotone and non-monotone areas based on block variances. This scheme is referred to as DFOVS (Deblocking Frames of Variable Size). A shortcoming of this approach is that its computational complexity is not suitable for real-time implementation and it causes blurring due to not fully distinguishing monotone from non-monotone areas. More recently, another approach called POCS (Projection Onto Convex Set) based on adaptive

projections has been proposed [2]. The idea behind POCS is to iteratively impose different sets of constraints to reduce the blocking artifacts. A set of constraints is applied as long as it is closed convex. In [2], three different sets of constraints are discussed: locally adaptive smoothness constraint, locally adaptive quantization constraint and intensity constraint. A shortcoming of this approach is that in order to get flexibility, a local threshold needs to be provided at a high computation cost. In the H.264 deblocking filter [4], deblocking is first applied vertically from left to right and then horizontally from top to bottom within each macroblock. The deblocking filter operation is done by initially computing the edge strength and then by providing a corresponding content dependent filtering operation. Experimental results from [6] show that the H.264/AVC adaptive deblocking filter is the least computationally complex because only add, shift and comparison operations are involved in its implementation. However, the gain obtained is not sufficient to meet the real-time decoding requirement. Further optimization steps are thus introduced in this paper with a tolerable level of video quality degradation.

In section II, an overview of the H.264 deblocking filter is first presented. The optimization steps introduced and their details are discussed in section III. Section IV includes the experimental results both in terms of video quality and computational complexity.

II. OVERVIEW OF H264/AVC DEBLOCKING FILTER

Since the H.264/AVC deblocking filter [4] is based on 4x4 blocks, four-line deblocking filtering is required across a boundary. For example, as shown in Figure 1, consider the boundary 1 across the two blocks (b,f) and (b,g). A separate deblocking operation is required for each line. As indicated in this figure, the deblocking order starts horizontally from left to right, and then vertically from top to bottom. First, the luminance component and then the chrominance components are processed.

The operation of the adaptive loop filter, which is one of the most computationally complex parts in the decoding process, can be partitioned into two stages: First computing the boundary strength parameter and then peforming content-dependent filtering. The purpose of the boundary strength stage, called Bs, is to check whether any blocking artifact is introduced on the boundary edge and then to decide the strength of the corresponding filter operation on this boundary. The boundary strength parameters or Bs are dependent on the intra/inter prediction, the existence of nonzero residue transform coefficients and the difference of motion vectors across the boundary. The decision flow for these parameters is illustrated in Table 1.

TABLE 1: DECISION OF BOUNDARY STRENGTH BASED ON CODED INFORMATION.

P and Q intra coded and boundary is macroblock boundary	Bs=4
P and Q intra coded and boundary is not macroblock boundary	Bs=3
Neither P and Q is intra coded, P or Q contain coded coefficients	Bs=2
Neither P and Q is intra coded, neither P and Q contain coded coefficients, P and Q have different reference frames or a different number of reference frames or different motion vectors.	Bs=1
Neither P and Q is intra coded, neither P and Q contain coded coefficients, P and Q have the same reference frame and identical motion vectors.	Bs=0

III. OPTIMIZATION STEPS FOR DEBLOCKING

In Table 1, the edge of the block copied directly from a previous frame without having different motion vectors or nonzero code coefficients is labelled by Bs=0, indicating there is no filtering applied on this boundary between two blocks. Such information has inspired to consider the skip mode adopted by H.264. To have skip modes, a macroblock should meet all of the following four conditions [7]:

- best motion compensation block size being 16x16;
- reference frame being just one previous;
- motion vector being (0,0);
- transform coefficients being all quantized to zero.

It is shown that the skip mode conditions of one macroblock allow the conditions for boundary strength to be equal to zero. In addition, before applying the H.264/AVC deblocking filter, the mode information for the current macroblock is obtained from previous encoding or decoding, and thus there is no extra computation for getting the mode information. So, our first optimization step is summarized in the following pseudo code labelled as optimization 1:

```
For (index=0; index<NumberMacroblock; index++)
{
        ModeInformation from Macroblock [index];
        If (modeInformation is skip mode)
                Continue; //avoid the deblocking.
```

```
Else
        Deblock_MB ();
}
```

As stated earlier, the deblocking filtering operation can be partitioned into two stages consisting of boundary strength computation and content-dependent filtering. Considering that each basic deblocking filtering unit is a 4x4 block, for each column of the horizontal deblocking filtering (dotted lines in Figure 3) or each row from the vertical deblocking filtering (thick lines in Figure 3), sixteen lines of boundary strength computations are required, labelled 1 through 16 in Figure 3, and followed by the content-dependent filtering operation. In the JM9.5 implementation of the H.264/AVC deblocking filter [4], the computation of boundary strength and filtering are separated, where the boundary strength is first computed and then the filtering is applied.

Our experimental results show that the original sixteen lines of the boundary strength computation for each column or each row takes more than 90% of the computation time of the entire deblocking filtering process. Due to the dominance of the boundary strength computation, we have attempted to optimize this computation. Since each adjacent block contains similar information including the motion vectors and intra/inter mode, we utilize such information to avoid any unnecessary boundary strength computation. For example, we only compute the boundary strength labelled as $1,3,5,\cdots$, to avoid computing $2,4,6,\cdots$, while we apply the filtering operation on these edges based on the boundary strength information of the previous one line from $1,3,5,\cdots$. Since in this approach, we only skip one line, leaving 8 lines to compute, we refer to this method as 8-line strength optimization. Similarly, we can skip four lines, referred to as 4-line strength optimization. If we skip eight lines, the remaining two lines to be computed is referred to as 2-line strength optimization. Similarly, only one line computation is referred to as 1-line strength optimization.

IV. EXPERIMENTAL RESULTS

In this paper, only QCIF video sequences are examined since such sequences are normally displayed on portable devices. In order to compare the computational complexity, several different measures are considered. Although actual time reduction as compared with the original algorithm can be used as a computational complexity measure, it is not informative enough because it depends on the processor and memory bandwidth utilized. Another measure is utilized here based on the instruction level information obtained from the *iprof* software [8]. This software provides the number and type of instructions. Our objective is to reduce the computational complexity of deblocking without scarifying video quality. The software *avsnr* [9] is a commonly used tool for obtaining a video quality comparison between two different coding algorithms.

In order to test the effectiveness of the introduced optimization 1, we randomly selected four different video clips varying in their characteristics from fast motion to low motion. The default encoding parameters were set as follows: fast motion estimation enabled, fast mode decision enabled, reference frames 5, motion estimation search range of 16, 150 frames for each video clip and the GOP format as $IPPP\cdots$. JM9.5 was selected as the reference software to run the H.264/AVC encoding/decoding. Table 2 provides the computational complexity and video quality comparison outcome for the first introduced optimization step. From this table, one can see that this step strongly depends on the number of skipped macroblocks before deblocking as part of the entire video decoding process. Since the "foreman" clip has more sharp motion regions compared with the other video clips, fewer number of skipped macroblocks was detected and therefore a lower time reduction was gained. Note in Table 2, *DF_time* represents the original deblocking time averaged over four different quantization parameters, *Opt time* represents the optimized deblocking time averaged over four different quantization parameters. Time reduction means (*DF_Time-Opt_Time)/DF_Time*; where *Percentage of skip_Num_MB* represents the ratio between the number of skipped macroblocks and the total number of macroblocks.

For testing the effectiveness of the second introduced optimization step, *iprof* was selected to provide the total number of instructions. This software provides the number and type of instructions without the need for performing any source code modification. The main instruction types include arithmetic, data (load and store), jump/test/compare, and rotate/shift instructions. It needs to be realized that the weight of an individual instruction type is dependent on a

specific processor. For the analysis reported here, the weights correspond to the Intel 686 processor; see Table 3, noting that they need to be appropriately adjusted depending on the processor utilized. By incorporating the weights of instruction types and the number of instructions, the total instruction cost was thus obtained.

TABLE 3: WEIGHTS OF INSTRUCTION TYPES FOR INTEL 686 PROCESSOR

Instruction types	Average weights
Arithmetic (add, subtract, etc.)	4
Data movement (load and store)	7
Rotate and shift	2
Jump and compare	5

Similarly we randomly selected four different QCIF format video clips for testing. Figure 4 shows the total number of instruction reduction for the three different representative videos. As can be seen in this figure, the 1-line strength optimization significantly reduced the deblocking filter computational complexity. A video quality comparison was also carried out based on the *avsnr* software and the results are reported in Table 4. Interestingly enough, the 8-line and 4-line optimization did not noticeably alter the video quality, and the 2-line and 1-line optimization had practically a small effect on the video quality.

V. CONCLUSION

In this paper, we have introduced two different optimization steps to reduce the computational complexity of the H264/AVC deblocking filter. The first optimization utilizes the skip mode information of decoded macroblocks by avoiding deblock filtering for some macroblocks. The second optimization is based on the similarity between adjacent blocks by avoiding unnecessary boundary strength computation. The benefits of these optimization steps lie in lowering the video decoding power consumption on portable devices.

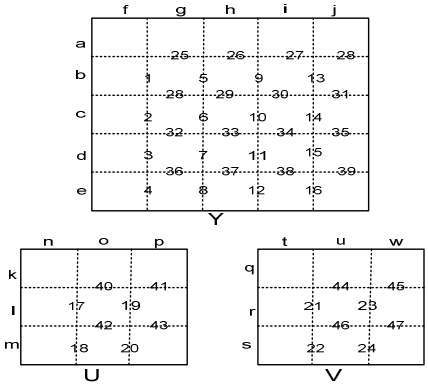

Figure1: H264/AVC deblocking filter order.

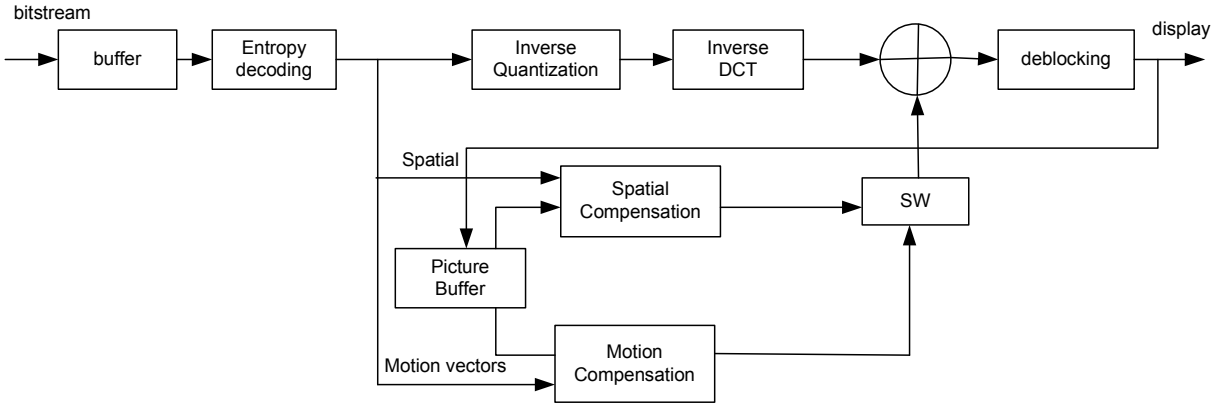

Figure 2: H.264/AVC decoding process block diagram [1].

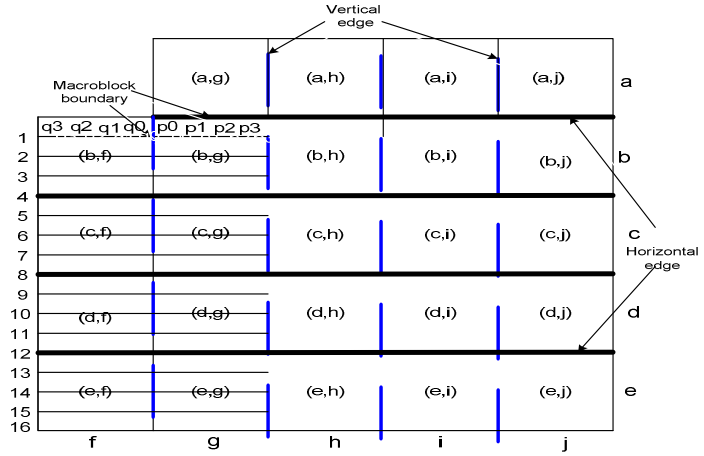

Figure 3: Illustration for adaptive loop deblocking filter operation in one macroblock.

TABLE 2: COMPUTATIONAL COMPLEXITY AND VIDEO QUALITY COMPARISON FOR THE FIRST OPTIMIZATION STEP .

Video Clips	Computational Complexity Analysis				Video quality comparison based on AVSNR[9], QP=22,27,32,37	
	DF_Time (ms)	Opt_Time (ms)	Time reduction	Percentage of Skip_Num_MB	DSNR(dB)	DBR
Claire	991	309	69%	81%	-0.005dB	0.12%
Container	1140	459	60%	78%	-0.044dB	1.02%
Foreman	1596	1449	9%	19%	0.020dB	-0.47%
Grandma	1070	404	62%	80%	0.005dB	-0.14%

TABLE 4: VIDEO QUALITY COMPARISON FOR DIFFERENT OPTIMIZATIONS IN BOUNDARY STRENGTH.

Video clips	QP=22,27,32,37							
	16 line Vs. 8-line		16 line Vs. 4-line		16 line Vs. 2- line		16 line Vs.1-line	
	DSNR	DBR	DSNR	DBR	DSNR	DBR	DSNR	DBR
Claire	0	0	0	0	-0.011	0.23%	-0.031	0.60%
Container	0	0	0	0	-0.002	0.02%	-0.036	0.83%
Foreman	0	0	0	0	-0.025	0.50%	-0.032	0.65%
grandma	0	0	0	0	-0.031	0.77%	-0.045	1.18%

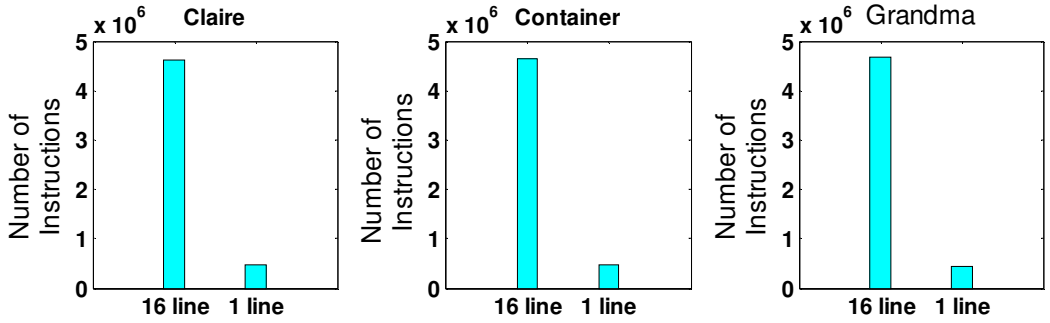

Figure 4: Instruction level comparison for computational complexity between un-optimized and optimized techniques for boundary strength.

REFERENCES

[1] M. Horowitz, A. Joch, F. Kossentini,etc. "H264/AVC Baseline Profile Decoder Complexity Analysis," *IEEE Transactions on Circuit and System for Video Technology*, Vol. 13, No.7, July 2003, pp.704-716.

[2] J. Zou and H. Yan, "A Deblocking Method for BDCT Compressed Images Based on Adaptive Projections," *IEEE Transaction on Circuits and Systems for Video Technology*, Vol. 15, No. 3, March 2005, pp.430-435.

[3] A. Averbuch, A. Schclar and D. Donoho, "Deblocking of Block-Transform Compressed Images Using Weighted Sums of Symmetrically Aligned Pixels," *IEEE Transactions on Image Processing*, Vol. 14, No. 62, Feb 2005, pp.200-212.

[4] P. List, A. Joch and J. Lainema. etc., "Adaptive Deblocking Filter," *IEEE Transactions on Circuits and Systems for Video Technology*, Vol. 13, No. 7, July 2003, pp.614-619.

[5] S. Tai, Y. Chen and S. Sheu, "Deblocking Filter for Low Bit Rate MPEG-4 Video," *IEEE Transactions on Circuits and Systems for Video Technology*, Vol. 15, No. 6, June 2005, pp.733-741.

[6] J. Ren and N. Ketharnarz, “A Power Consumption Comparison study of Motion Compensation and Deblocking Filters for Video Coding,” submitted ICIP 2007.
[7] Joint Video Team of ISO/IEC MPEG&ITU-T VCEG,” Draft ITU-T Recommendation and Final Draft International Standard of Joint Video Specification, Doc.G050R1, March, 2003.
[8] P. Kuhn, Algorithms, Complexity Analysis and VLSI Architecture for MPEG-4 Motion Estimation, Kluwer Academic Publishers, 2003.
[9] G. Bjontegaard,”Calculation of Average PSNR Differences between RD-curves,” Doc. VCEG-M33, April, 2001.

Jianfeng Ren received his BS degree from Xi’an University of Technology, P.R.China, in 2000 and MS degree from NorthWestern Polytechnic University, P.R.China, in 2003. Currently, he is pursuing his PhD in Electrical Engineering at the University of Texas at Dallas. His research interests include video coding and real-time image processing, particularly interested in low-power video coding.

Nasser Kehtarnavaz (S’82-M’86-SM’92) received the PhD degree from Rice University, Houston, TX, in 1987. He is currently a professor with the Department of Electrical Engineering and Director of the Signal and Image Processing Lab at the University of Texas at Dallas. He was previously a professor of electrical engineering at Texas A&M University. His research interests include digital signal and image processing, real-time image processing, and pattern recognition. He has authored or co-authored five books and numerous articles as related to these areas. Dr. Kehtarnavaz is currently serving as the Chair of the Dallas Chapter of the IEEE Signal Processing Society.